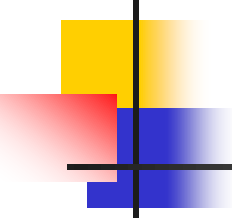


Gestion des évènements

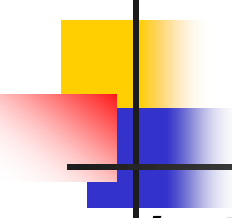


Dr. Khadim Dramé
Khadim.drame@univ-zig.sn
Département Informatique
Université Assane Seck de Ziguinchor



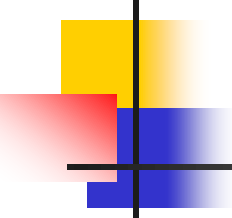
Plan du cours

- Introduction
- Évènements
- Écouteurs
- Traitement d'évènements



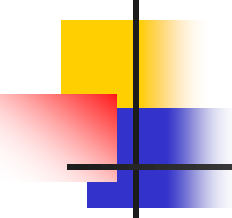
Notion d'événement

- L'utilisation de GUI impose une façon particulière de programmer: **programmation événementielle** (*event-driven programming*)
- **Évènement** = changement d'état d'un objet
 - créé soit directement, soit par une action de l'utilisateur (clic, glissement de la souris,...)
- Un évènement peut déclencher une action à réaliser par des composants
- Le composant déclencheur de l'évènement est appelé **source**
- Le composant qui traite l'évènement est appelé **auditeur** ou **écouteur** (*listener*)



Gestion des événements en Java

- De nombreuses classes et interfaces pour gérer les événements
- Définies dans les packages `java.awt.event` et `javax.swing.event`
- Les classes *event* héritent de `java.awt.event.AWTEvent`
- Les interfaces *Listener* associées aux événements héritent de `java.util.EventListener`



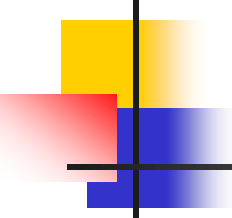
Types d'événements

■ Événements de **bas niveau**

- **MouseEvent** (appuyer, relâcher, déplacer le pointeur de la souris)
- **KeyEvent** (appuyer, relâcher une touche clavier)
- **WindowEvent** (fermer une fenêtre)
- **FocusEvent** (focus clavier)

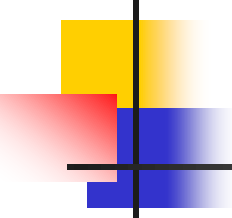
■ Événements de **haut niveau**

- **ActionEvent** (cliquer sur un bouton, un champ textuel)
- **TextEvent** (modifier le texte entré)
- **ItemEvent** (sélectionner un item)
- ...



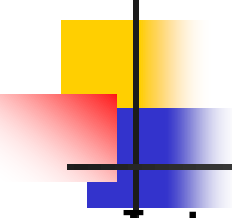
ActionEvent

- Cette classe décrit des évènements de **haut niveau** qui vont souvent déclencher des réactions
 - clic sur un bouton
 - *Entrée* dans un champs de texte
 - choix dans une liste
 - choix dans un menu
- Ces évènements sont couramment utilisés et simples à traiter



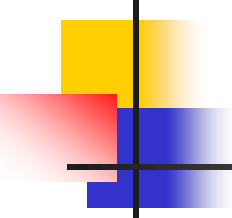
Les écouteurs

- Les gestionnaires d'évènements, appelés **auditeurs** (*listeners*)
- Ils implémentent une interface qui définit une ou plusieurs méthodes de gestion d'évènements
- Ces méthodes sont invoquées chaque fois qu'un événement se produit
- Les auditeurs ont une *source* d'évènements, généralement un composant
- Un auditeur est associé à un type d'évènement



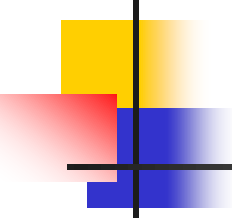
Les écouteurs

- Interfaces *Listener* associées aux évènements
 - *MouseListener*
 - *MouseMotionListener*
 - *KeyListener*
 - *WindowListener*
 - *ActionListener*
 - *TextListener*
 - *ItemListener*
 - ...
- Ces interfaces doivent être implémentées
- L'implémentation consiste à redéfinir toutes les méthodes ou les laisser vides



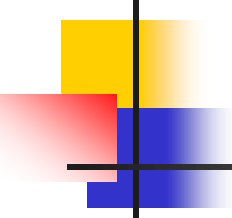
ActionListener, TextListener

- *ActionListener*, auditeur associé aux événements de type *ActionEvent*
 - une seule méthode à implémenter:
 - void **actionPerformed**(ActionEvent e)
 - un auditeur de ce type peut être associé à plusieurs types de composants (boutons, champs textuels, entrées de menus)
- *TextListener*, auditeur associé aux *TextEvent*
 - une seule méthode
 - void **textValueChanged**(TextEvent e)



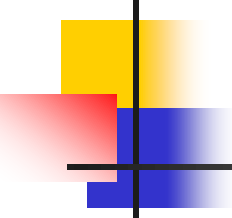
MouseListener, MouseMotionListener

- Des auditeurs associés aux *MouseEvent*
 - *MouseListener*
 - Méthodes
 - void **mousePressed**(MouseEvent e)
 - void **mouseReleased**(MouseEvent e)
 - void **mouseClicked**(MouseEvent e)
 - void **mouseEntered**(MouseEvent e)
 - void **mouseExited**(MouseEvent e)
 - *MouseMotionListener*
 - Méthodes
 - void **mouseDragged**(MouseEvent e)
 - void **mouseMoved**(MouseEvent e)



KeyListener, WindowListener

- *KeyListener*, auditeur associé aux *KeyEvent*
 - Méthodes
 - void **keyPressed**(KeyEvent e)
 - void **keyReleased**(KeyEvent e)
 - void **keyTyped**(KeyEvent e)
- *WindowListener*, auditeur associé aux *WindowEvent*
 - Méthodes
 - void **windowActivated**(WindowEvent e)
 - void **windowClosed**(WindowEvent e)
 - void **windowClosing**(WindowEvent e)
 - ...

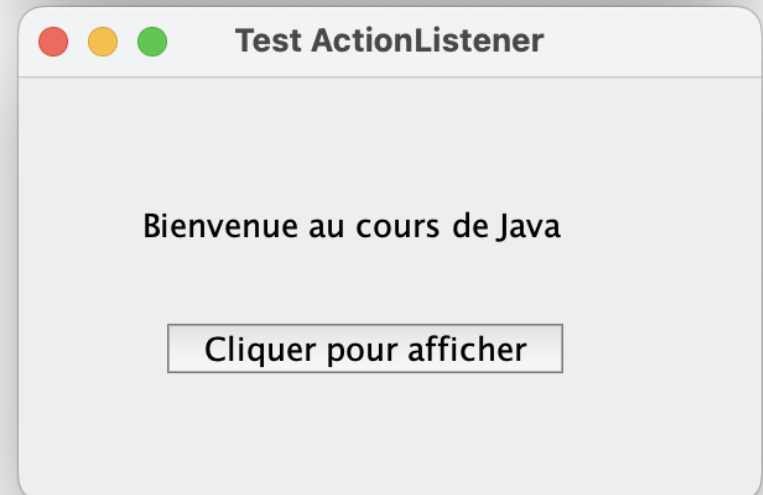
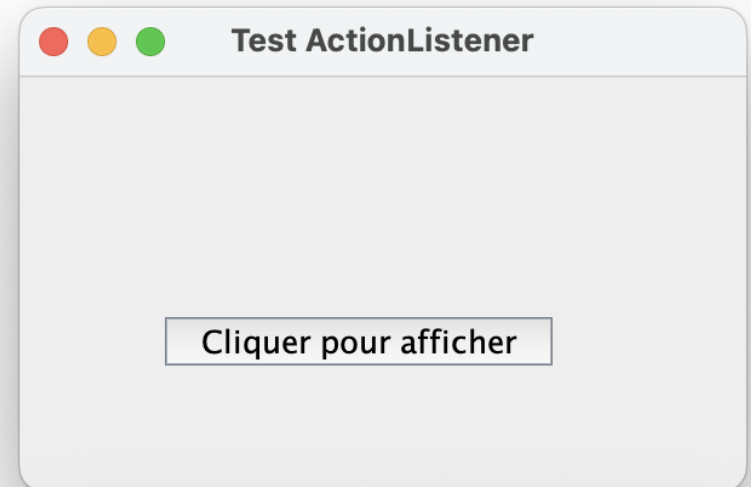


Associer un écouteur à un composant

- Les événements sont créés par les composants
 - un composant peut avoir un ou plusieurs écouteurs
 - un même écouteur peut être associé à plusieurs composants
- Pour associer un écouteur à un composant
 - utiliser la méthode: `void addXxxListener(XxxListener)`
 - Exemples
 - `addActionListener(ActionListener)`
 - `addMouseListener(MouseListener)`
 - `addTextListener(TextListener)`
- L'écouteur doit redéfinir les méthodes de l'interface correspondante
- L'écouteur s'intéresse ainsi aux événements correspondants engendrés par le composant

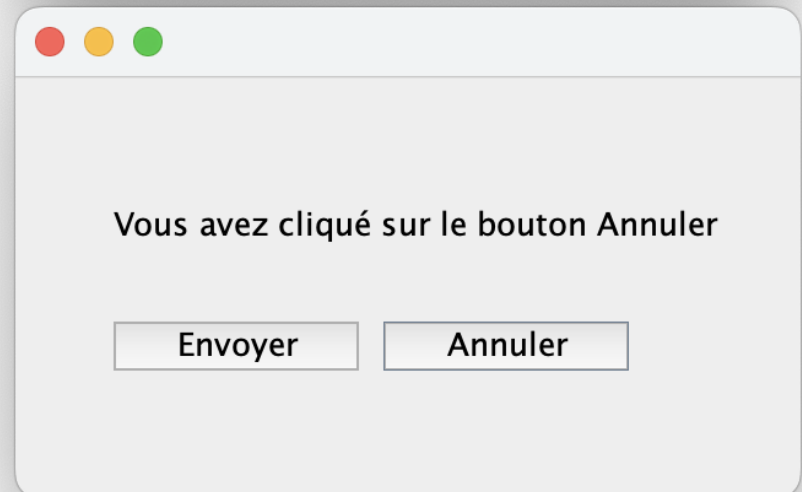
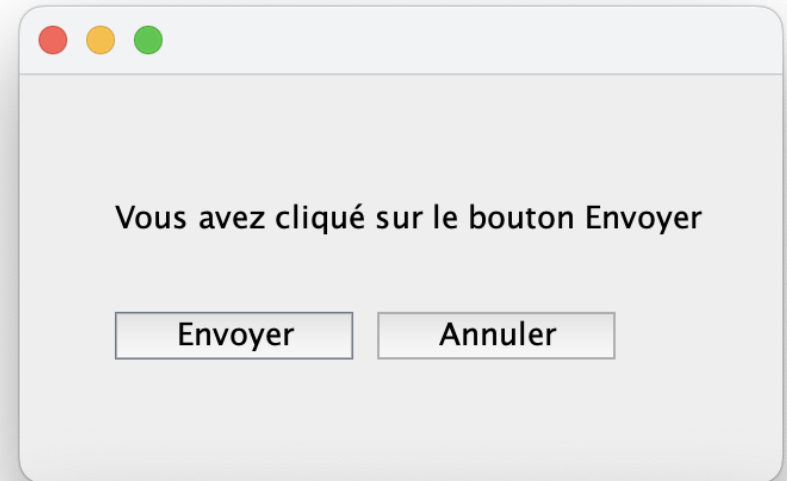
Exemple 1 (ActionListener avec un bouton)

```
package sn.uasz.m1.gui.listener;
import javax.swing.*;
import java.awt.event.*;
public class BoutonListener extends JFrame
    implements ActionListener {
    private JLabel label;
    public BoutonListener() {
        setTitle("Test ActionListener");
        label = new JLabel();
        label.setBounds(40,50,170,20);
        JButton bouton = new JButton("Afficher message");
        bouton.setBounds(50,100,140,20);
        bouton.addActionListener(this);
        add(bouton);
        add(label);
        setSize(300,200);
        setDefaultCloseOperation(EXIT_ON_CLOSE);
        setVisible(true);
    }
    public static void main(String[] args) {
        BoutonListener fen = new BoutonListener();
        fen.setLayout(null);
    }
    //implémentation de la méthode actionPerformed
    public void actionPerformed(ActionEvent e){
        label.setText("Bienvenue au cours de Java");
    }
}
```



Exemple 2 (méthode getSource())

```
public class TwoBoutonListener extends JFrame implements
    ActionListener {
    private JLabel label;
    private JButton envoyer, annuler;
    public TwoBoutonListener() {
        label = new JLabel();
        label.setBounds(40,50,170,20);
        envoyer = new JButton("Envoyer");
        envoyer.setBounds(40,100,100,20);
        annuler = new JButton("Annuler");
        annuler.setBounds(150,100,100,20);
        //action associée aux boutons
        envoyer.addActionListener(this);
        annuler.addActionListener(this);
        add(envoyer); add(annuler); add(label);
        setSize(300,200);
    }
    public static void main(String[] args) {
        TwoBoutonListener fen = new TwoBoutonListener();
        fen.setLayout(null); fen.setVisible(true);
    }
    //implémentation de la méthode actionPerformed
    public void actionPerformed(ActionEvent e){
        if(e.getSource() == envoyer){
            label.setText("Action sur bouton Envoyer");
        } else if(e.getSource() == annuler){
            label.setText("Action sur bouton Annuler");
        }
    }
}
```





Exemple 3 (classes internes)

```
package sn.uasz.m1.gui.listener;
import javax.swing.*;
import java.awt.event.*;
public class TwoBoutonListener2 extends JFrame {
    private JLabel label;
    private JButton envoyer, annuler;
    public TwoBoutonListener2() {
        label = new JLabel();
        label.setBounds(40,50,170,20);
        envoyer = new JButton("Envoyer");
        envoyer.setBounds(40,100,100,20);
        annuler = new JButton("Annuler");
        annuler.setBounds(150,100,100,20);
        //actions associées aux boutons
        envoyer.addActionListener(new
            EnvoyerListener());
        annuler.addActionListener(new
            AnnulerListener());
        add(envoyer); add(annuler); add(label);
        setSize(300,200);
    }
    public static void main(String[] args) {
        TwoBoutonListener2 fen = new
            TwoBoutonListener2();
        fen.setLayout(null);
        fen.setVisible(true);
        fen.setDefaultCloseOperation(EXIT_ON_CLOSE)
    }
}
```

```
//utilisation de classes imbriquées
class EnvoyerListener implements ActionListener {
    public void actionPerformed(ActionEvent e){
        label.setText("Action sur bouton Envoyer");
    }
}
class AnnulerListener implements ActionListener {
    public void actionPerformed(ActionEvent e){
        label.setText("Action sur bouton Annuler");
    }
}
} //fin de la classe principale
```



Exemple 4 (classes internes anonymes)

```
package sn.uasz.m1.gui.listener;
public class TwoBoutonListener3 extends JFrame {
    private JLabel label;
    private JButton envoyer, annuler;
    public TwoBoutonListener2() {
        label = new JLabel();
        label.setBounds(40,50,170,20);
        envoyer = new JButton("Envoyer");
        envoyer.setBounds(40,100,100,20);
        annuler = new JButton("Annuler");
        annuler.setBounds(150,100,100,20);
        //actions associées aux boutons
        envoyer.addActionListener(new ActionListener(){
            @Override
            public void actionPerformed(ActionEvent e) {
                label.setText("Action sur bouton Envoyer");
            }
        });
        annuler.addActionListener(new ActionListener(){
            @Override
            public void actionPerformed(ActionEvent e) {
                label.setText("Action sur bouton Annuler");
            }
        });
        add(envoyer); add(annuler); add(label);
        setSize(300,200);
    }
}
```

```
public static void main(String[] args) {
    TwoBoutonListener3 fen = new
        TwoBoutonListener3();
    fen.setLayout(null);
    fen.setVisible(true);
    fen.setDefaultCloseOperation(EXIT_ON_CLOSE)
}
}
```




Exemple 5 (ItemListener)

```
package sn.uasz.m1.gui.listener;
import javax.swing.JComboBox;
import javax.swing.JFrame;
import javax.swing.JLabel;
import java.awt.FlowLayout;
import java.awt.event.ItemEvent;
import java.awt.event.ItemListener;

public class TestItemListener extends JFrame implements ItemListener {
    private JComboBox<String> combo;
    JLabel label = new JLabel("Valeur par défaut");
    public TestItemListener() {
        String choix[] = {"Enseignant", "Médecin", "Infirmier", "Gendarme", "Magistrat"};
        combo = new JComboBox<String>(choix);
        combo.addItemListener(this);
        add(combo);
        add(label);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setSize(300, 200);
        setVisible(true);
        getContentPane().setLayout(new FlowLayout());
    }
    public void itemStateChanged(ItemEvent e) {
        label.setText((String)combo.getSelectedItem());
    }
    public static void main(String[] arg) {
        new TestItemListener();
    }
}
```

Exercice d'application

- Créer une fenêtre ayant trois composants: un JLabel qui affiche le texte « Compteur = 0 » en haut et deux JButton avec des textes Incrémenter et Décrémenter en bas (Cf Figure).
- Ajouter des événements aux boutons qui permettront d'incrémenter (respectivement de décrémenter) la valeur de compteur si l'on clique sur le bouton Incrémenter (respectivement sur le bouton Décrémenter).

